

OOP

UNIT 7 – INHERITANCE

A derived class does not repeat its parent's story. It continues it.

FROM ONE CLASS TO A FAMILY OF CLASSES

In the previous units, the Song class was gradually extended. It stores a title, an artist, and a duration, and its objects can now be copied, compared, combined, and printed like built-in values.

A real music application, however, rarely works with only one kind of audio item. It may need a LiveRecording, which is a song with a venue and an audience size, or a Podcast, which has an episode number and a host. These types need *some* of the same information and operations, but each also *needs additional information and/or specialized behavior*.

THE PROBLEM OF DUPLICATION

The most direct approach is to write every type as a completely independent class. LiveRecording and Podcast would then repeat fields such as title and duration, as well as the functions that print or examine them. This compiles, but it spreads the same idea across several classes.

Duplicated code is difficult to maintain. Fixing a bug in the shared logic, changing the way duration is validated, or adding a new common field now requires several separate changes. The related classes may gradually become inconsistent.

REUSING A CLASS THROUGH INHERITANCE

Inheritance allows one class to be defined in terms of another. A LiveRecording can be declared as a specialized Song that adds only the venue and audience size. It *reuses* the existing public behavior instead of repeating it.

In this unit, we build inheritance from the ground up. We examine class hierarchy syntax, access control, construction and destruction, overriding, virtual functions, polymorphism, virtual destructors, object slicing, and the design choice between inheritance and composition.

LEARNING OBJECTIVES

- Explain why inheritance exists and when it models a genuine is-a relationship.
- Use the class hierarchy syntax and public and protected access specifiers.
- Trace the order in which base and derived constructors and destructors run.
- Initialize a base class correctly from a derived class's constructor initializer list.
- Distinguish overriding a virtual function from hiding a non-virtual one.
- Use the virtual keyword and the override keyword.
- Explain why a base class meant for polymorphic use needs a virtual destructor.
- Explain object slicing and why it happens.
- Decide when inheritance is the right tool for a design and when composition is a better fit.

INHERITANCE AND BASE/DERIVED CLASSES

Inheritance models an *is-a* relationship between classes. A LiveRecording is a Song: reading its title, examining its duration, or printing its song information still makes sense. The LiveRecording simply provides additional information.

This differs from a *has-a* relationship. A Playlist has Song objects, but a Playlist is not a Song. The first and most important design decision is therefore not syntax. It is deciding whether the proposed relationship is genuinely is-a.

In C++, the class that supplies common data and behavior is called the *base* class. A class that *specializes or extends* it is called the *derived* class.

```
class Song                      // Base class
{
    // Common song data and behavior
};

class LiveRecording : public Song // Derived class
{
    // Additional live-recording data and behavior
};
```

Read the declaration aloud:

LiveRecording publicly inherits from Song, or more simply: LiveRecording **is a** Song.

A LiveRecording needs the title, artist, and duration that Song already stores, *plus* a venue and an audience size. Declaring LiveRecording as a derived class of Song is written as `class LiveRecording : public Song`.

The inheritance itself is public: every public member of Song is public in LiveRecording, which is the correct choice when the *derived* class genuinely is a kind of the *base* class.

Song's constructor takes three parameters and has no default constructor, so a LiveRecording object's initializer list calls Song's constructor by name, exactly as it would initialize an ordinary data member.

```
class Song {
private:
    string title;
    string artist;
    int duration;
public:
    Song(string title, string artist, int duration)
        : title(title), artist(artist), duration(duration) { }
    string getTitle() const { return title; }
    string getArtist() const { return artist; }
    int getDuration() const { return duration; }
};
```

```

class LiveRecording : public Song {
private:
    string venue;
    int audienceSize;

public:
    LiveRecording(string title, string artist, int duration, string venue,
int audienceSize)
        : Song(title, artist, duration), venue(venue),
audienceSize(audienceSize)    {    }

    string getVenue() const { return venue; }
    int getAudienceSize() const { return audienceSize; }
};

int main() {
    LiveRecording show("Fix You", "Coldplay", 295,
                        "Wembley Stadium", 90000);
    cout << show.getTitle() << " by " << show.getArtist()
        << " (" << show.getDuration() << "s)" << endl;
    cout << "Recorded at " << show.getVenue()
        << " for " << show.getAudienceSize() << " people" << endl;
    return 0;
}

```

The program prints:

```
Fix You by Coldplay (295s)  
Recorded at Wembley Stadium for 90000 people
```

getTitle, getArtist, and getDuration are never defined in LiveRecording.

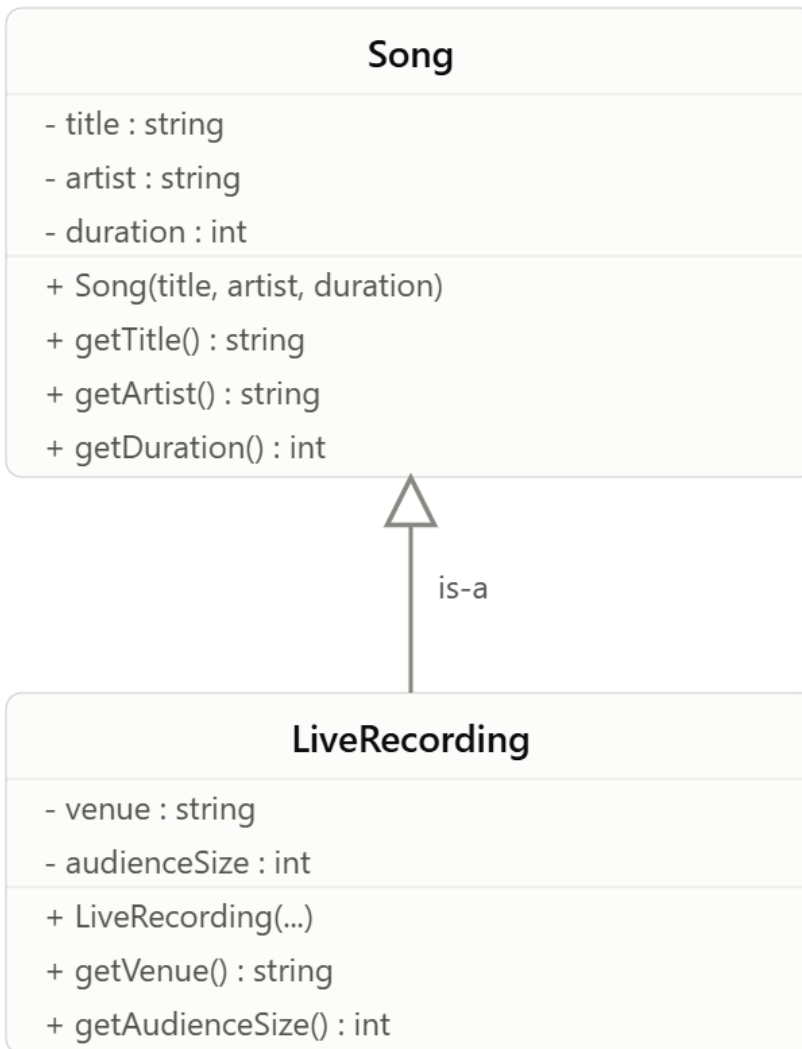
They are *inherited*: because LiveRecording is publicly derived from Song, every public member function of Song is directly callable on a LiveRecording object, as if it had been declared there. getVenue and getAudienceSize are new members that only LiveRecording has. A LiveRecording object physically contains a complete Song object as its base part, plus the venue and audienceSize fields declared in LiveRecording itself.

COMMON MISTAKE: FORGETTING TO INITIALIZE THE BASE CLASS

Song has no default constructor, so a derived class's initializer list must call one of Song's constructors explicitly whenever it takes arguments. Omitting that call does not silently fall back to default values -- it fails to compile, because the compiler has no way to construct the Song part of the object.

```
class LiveRecording : public Song {
private:
    string venue;
    int audienceSize;
public:
    // Mistake:
    // Song has no default constructor, so the compiler cannot build the
    // base part of the object and this fails to compile.
    LiveRecording(string title, string artist, int duration, string venue,
int audienceSize)
        : venue(venue), audienceSize(audienceSize) {
    }
    string getVenue() const { return venue; }
};
```

This produces a real compiler error, no matching function for call to Song::Song()



ACCESS SPECIFIERS AND INHERITANCE

LiveRecording now wants a convenience method, isLong, that checks whether the recording runs longer than five minutes. The obvious implementation compares duration directly, but duration is a private member of Song.

```
class Song {
private:
    string title;
    string artist;
    int duration;
public:
    Song(string title, string artist, int duration)
        : title(title), artist(artist), duration(duration)    {    }
};

class LiveRecording : public Song {
private:
    string venue;
public:
    LiveRecording(string title, string artist, int duration, string venue)
        : Song(title, artist, duration), venue(venue)    {    }
```

```

bool isLong() const
{
    // Mistake: duration is private in Song, so it is not visible here,
    // even though LiveRecording is a derived class.

    return duration > 300;
}

};

int main()
{
    LiveRecording show("Reverie", "Debussy Ensemble", 900, "Carnegie
Hall");
    cout << show.isLong() << endl;
    return 0;
}

```

This fails to compile with `int Song::duration` is private within this context. Being a derived class grants access to the base class's public interface, but not to its private implementation details -- private truly means private to that one class, even from its own subclasses. This surprises many beginners, who expect "derived" to mean "can see everything."

C++ provides a third access level for exactly this situation: *protected*. A protected member is inaccessible from outside the class, exactly like a private one, but it remains visible inside any class derived from it. Changing Song's data members to protected lets LiveRecording read them directly.

```
#include <iostream>
#include <string>

using namespace std;

class Song {
protected:
    string title;
    string artist;
    int duration;

public:
    Song(string title, string artist, int duration)
        : title(title), artist(artist), duration(duration) { }
    string getTitle() const { return title; }
};
```

```

class LiveRecording : public Song {
private:
    string venue;
    static const int longsz = 300;
public:
    LiveRecording(string title, string artist, int duration, string venue)
        : Song(title, artist, duration), venue(venue) { }

    // duration is protected in Song, so LiveRecording can read it
    directly.
    bool isLong() const {
        return duration > longsz;
    }
};

int main() {
    LiveRecording show("Fix You", "Coldplay", 295,
        "Wembley Stadium");

    cout << show.getTitle() << " is long: " << (show.isLong()? "yes" :
    "no") << endl;
    return 0;
}

```

The program prints: Fix You is long: no

isLong now compiles because duration is protected rather than private, and protected members are exactly as visible to a derived class's member functions as private members are to the base class's own member functions. Nothing about the class's public interface has changed -- getTitle is still the only way for outside code to read the title -- only the boundary between the base class and its descendants has moved.

Base member access	Visible in base class	Visible in derived class	Visible outside the hierarchy
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

COMMON MISTAKE: TREATING PROTECTED AS PUBLIC

protected widens access for a chosen family of classes, not for the rest of the program. Code outside the class hierarchy still cannot reach a protected member .

```
class Song
{
protected:
    string title;
```

```

    int duration;

public:
    Song(string title, int duration) : title(title), duration(duration) {}
};

int main()
{
    Song track("Reverie", 900);

    // Mistake: protected members are visible to derived classes, not to
    // ordinary code outside the class hierarchy. This still fails to
compile.
    cout << track.duration << endl;

    return 0;
}

```

This fails to compile, with `int Song::duration` is *protected* within this context, because `main` is not part of the `Song` hierarchy. *protected* is best reserved for members a family of related classes genuinely needs to share; every protected member is a small weakening of encapsulation, since any future derived class can depend on it. When a plain accessor (getter) function would do the job, prefer it over widening access.

ORDER OF CONSTRUCTION AND DESTRUCTION

A LiveRecording object contains a Song *subobject* plus its own venue field. Before any of LiveRecording's own initialization can safely happen, the Song part it builds on must already exist -- there is no such thing as a partially-constructed base class supporting a derived class on top of it. C++ enforces this directly: the base class's constructor always finishes running before the derived class's constructor body begins, and destruction happens in the exact opposite order.

```
class Song
{
protected:
    string title;
public:
    Song(string title) : title(title)    {
        cout << "Song constructed: " << title << endl;
    }
    ~Song() {
        cout << "Song destroyed: " << title << endl;
    }
};
```

```

class LiveRecording : public Song {
private:
    string venue;

public:
    LiveRecording(string title, string venue)
        : Song(title), venue(venue) {
        cout << "LiveRecording constructed: " << venue << endl;
    }

    ~LiveRecording() {
        cout << "LiveRecording destroyed: " << venue << endl;
    }
};

int main()
{
    {
        LiveRecording show("Reverie", "Carnegie Hall");
    }
    return 0;
}

```

The program prints:

```
Song constructed: Reverie  
LiveRecording constructed: Carnegie Hall  
LiveRecording destroyed: Carnegie Hall  
Song destroyed: Reverie
```

Construction runs base-to-derived: Song's constructor body finishes first, so the inherited part of the object is always ready before LiveRecording adds anything to it. Destruction runs derived-to-base, the mirror image: LiveRecording's destructor runs first, while the Song part is still fully intact and usable, and only afterward is the Song part destroyed. This ordering is not a convention -- it is guaranteed by the language, and no amount of rearranging code in the derived class can change it.

COMMON MISTAKE: TRUSTING THE INITIALIZER LIST'S WRITTEN ORDER

The base class is always initialized *first*, and data members are always initialized in the order they are declared in the class, regardless of the order they appear in the initializer list.

```
class Song {
protected:
    string title;
public:
    Song(string title) : title(title)    {
        cout << "Song constructed: " << title << endl;
    }
};
class LiveRecording : public Song {
private:
    string venue;
public:
    // Mistake: listing venue before Song(title) in the initializer list
    // does not change the execution order. The base class is initialized
    // first, so the compiler warns that the listed order is misleading.
    LiveRecording(string title, string venue)
        : venue(venue), Song(title)
    {

```

```
        cout << "LiveRecording constructed: " << venue << endl;
    }
};
int main()
{
    LiveRecording show("Reverie", "Carnegie Hall");
    return 0;
}
```

This compiles, but the compiler issues a warning: `LiveRecording::venue` will be initialized *after* base `Song`, warning it that the listed order does not match the real order. Running the program confirms it -- `Song` is still constructed first, exactly as before, no matter where `Song(title)` appears in the list. Writing the initializer list in the true execution order, base class first and then members in declaration order, avoids the warning and keeps the code honest about what actually happens.

OVERRIDING MEMBER FUNCTIONS

Song already prints a short description of a track. LiveRecording wants its own version that also mentions the venue, so it defines a describe function with the same name.

```
class Song {
protected:
    string title;
public:
    Song(string title) : title(title) {}
    void describe() const {
        cout << "Song: " << title << endl;
    }
};

class LiveRecording : public Song {
private:
    string venue;
public:
    LiveRecording(string title, string venue)
        : Song(title), venue(venue) {}
    void describe() const {
        cout << "LiveRecording: " << title << " at " << venue << endl;
    }
};
```

```
void announce(const Song& track) {  
    track.describe();  
}  
  
int main() {  
    LiveRecording show("Reverie", "Carnegie Hall");  
  
    show.describe();    // calls LiveRecording::describe directly  
    announce(show);     // surprisingly calls Song::describe  
  
    return 0;  
}
```

The program prints:

```
LiveRecording: Reverie at Carnegie Hall  
Song: Reverie
```

Calling `show.describe()` directly gives the expected `LiveRecording` version, since `show`'s declared type and its real type are both `LiveRecording`.

But `announce` takes its argument as a `const Song&`, and inside `announce`, `track.describe()` calls `Song::describe` -- even though the object passed in is genuinely a `LiveRecording`. The compiler chooses which `describe` to call based on the declared type of the reference, `track`, not on the actual object it refers to.

`LiveRecording`'s `describe` does not override `Song`'s; it simply *hides* it, and hiding only works when code accesses the object through its own most-derived type.

THE VIRTUAL KEYWORD AND DYNAMIC BINDING

The behavior above defeats the entire point of writing `announce` to accept any `Song`: it should describe whatever track it is actually given, live recording, podcast, or plain song alike, without needing a separate function for each. Marking `Song`'s `describe` function `virtual` tells the compiler to choose which override to call based on the object's real type at run time, not on the declared type of the reference or pointer used to reach it. This is called dynamic binding, in contrast to the static binding used for ordinary function calls.

A derived class's overriding function should be marked `override`. This is not required to make overriding work, but it asks the compiler to check that a matching virtual function actually exists in the base class, turning a whole category of silent mistakes -- covered later in this section -- into compile errors instead.

```
class Song {
protected:
    string title;
    string artist;
public:
    Song(string title, string artist) : title(title), artist(artist) {}

    virtual void describe() const {
        cout << title << " by " << artist << endl;
    }
};
```

```

class LiveRecording : public Song {
private:
    string venue;

public:
    LiveRecording(string title, string artist, string venue)
        : Song(title, artist), venue(venue) { }

    virtual void describe() const override {
        cout << title << " by " << artist
            << " -- recorded live at " << venue << endl;
    }
};

class Podcast : public Song {
private:
    int episodeNumber;

public:
    Podcast(string title, string host, int episodeNumber)
        : Song(title, host), episodeNumber(episodeNumber) { }
}

```

```

    virtual void describe() const override {
        cout << title << " -- episode " << episodeNumber
            << ", hosted by " << artist << endl;
    }
};

void announce(const Song& track) {
    track.describe();
}

int main()
{
    Song studio("Imagine", "John Lennon");
    LiveRecording live("Reverie", "Debussy Ensemble", "Carnegie Hall");
    Podcast episode("Deep Focus", "Ana Reyes", 42);

    announce(studio); //song describe
    announce(live); // live recording
    announce(episode); //podcast

    return 0;
}

```

The program prints:

```
Imagine by John Lennon  
Reverie by Debussy Ensemble -- recorded live at Carnegie Hall  
Deep Focus -- episode 42, hosted by Ana Reyes
```

announce is written once, against the Song interface, and never mentions LiveRecording or Podcast by name. Yet calling it with a Song, a LiveRecording, and a Podcast produces three different, correct descriptions. Each object carries enough information at run time for the call to describe to be resolved against its real type, not the reference type used to pass it around. This is the essential idea behind polymorphism: one interface, many correct behaviors, selected automatically based on what the object actually is.

without virtual : static binding =>

- compiler binds function call to definition during compilation

with virtual : dynamic binding ->

- compiler does not binds the function call to the definition

COMMON MISTAKE: CALLING A VIRTUAL FUNCTION FROM A CONSTRUCTOR

Dynamic dispatch relies on the object's real type being fully established, and during a base class's own constructor, the derived part of the object does not exist yet. Calling a virtual function from a base class constructor therefore always resolves to the base class's own version, never to an override defined further down the hierarchy, even when constructing a derived object.

```
class Song
{
protected:
    string title;

public:
    Song(string title) : title(title)
    {
        announce(); // called while the object is still only a Song
    }

    virtual void announce() const
    {
        cout << "Song: " << title << endl;
    }
};
```

```

class LiveRecording : public Song
{
public:
    LiveRecording(string title) : Song(title)
    {
    }

    void announce() const override
    {
        cout << "LiveRecording: " << title << endl;
    }
};

int main()
{
    // The output is "Song: Reverie", not "LiveRecording: Reverie".
    // During Song's constructor, the LiveRecording part of the object
    // does not exist yet, so virtual dispatch resolves to Song::announce.
    LiveRecording show("Reverie");
    return 0;
}

```

The program prints Song: Reverie, not LiveRecording: Reverie, even though the object being constructed is a LiveRecording. While Song's constructor is running, LiveRecording has not been built yet, so there is no LiveRecording override to dispatch to -- the object is, for the moment, genuinely just a Song. This is a well-known and easy-to-miss pitfall: a base class constructor or destructor should never rely on a derived class's override running in its place.

OVERRIDING VS. HIDING, AND THE OVERRIDE KEYWORD

A derived function only overrides a base class's virtual function if its signature matches exactly: same name, same parameter types, and the same const-qualification. Any mismatch means the derived function hides the base version instead of overriding it, silently reproducing the exact hiding problem seen earlier, even though describe was declared virtual.

```
class Song
{
protected:
    string title;

public:
    Song(string title) : title(title) {}

    virtual void describe() const
    {
        cout << "Song: " << title << endl;
    }
};

class LiveRecording : public Song
{
```

```

public:
    LiveRecording(string title) : Song(title) {}

    // Mistake: missing const. The signature no longer matches
    // Song::describe() const, so this does not override it -- it hides it.
    // The override keyword turns that silent mistake into a compile error.
    void describe() override
    {
        cout << "LiveRecording: " << title << endl;
    }
};

int main()
{
    LiveRecording show("Reverie");
    show.describe();
    return 0;
}

```

LiveRecording's describe is missing the const that Song::describe() const declares, so the two signatures do not match. Without the override keyword, this would compile silently and quietly hide Song's version, reintroducing the very bug the virtual keyword was meant to fix. Because describe is marked override here, the compiler instead reports

LiveRecording::describe marked override, but does not override, catching the mistake immediately at the point where it was made.

COMMON MISTAKE: OMITTING OVERRIDE AND LOSING THE SAFETY NET

The fix in the example above is not to remove override -- it is to correct the signature so it genuinely matches the base class's virtual function. Marking every intended override with the override keyword costs nothing at run time and converts an entire class of hard-to-notice bugs into immediate compile errors. There is rarely a good reason to omit it.

VIRTUAL DESTRUCTORS

LiveRecording needs to record which seats were occupied, so it allocates an array of attendee identifiers on the heap and releases it in its destructor. A cleanup function, retire, is written once against the Song interface so it can retire any kind of track through a Song pointer.

```
class Song
{
protected:
    string title;

public:
    Song(string title) : title(title) {}

    virtual void describe() const    {
        cout << "Song: " << title << endl;
    }

    ~Song()    {
        cout << "Song destroyed: " << title << endl;
    }
};
```

```

class LiveRecording : public Song
{
private:
    int* attendeeIds;

public:
    LiveRecording(string title, int audienceSize)
        : Song(title), attendeeIds(new int[audienceSize])    { }

    void describe() const override
    {
        cout << "LiveRecording: " << title << endl;
    }

    ~LiveRecording()    {
        delete[] attendeeIds;
        cout << "LiveRecording attendee records released" << endl;
    }
};

```

```
int main()
{
    Song* show = new LiveRecording("Reverie", 1200); //upcasting
    // some code goes here
    delete show;

    return 0;
}
```

The compiler warns: deleting object of polymorphic class type Song which has non-virtual destructor might cause undefined behavior. Running the program shows exactly what that means -- it prints only Song destroyed: Reverie. LiveRecording attendee records released never appears, and the attendeeIds array is never freed.

COMMON MISTAKE: A NON-VIRTUAL DESTRUCTOR IN A POLYMORPHIC BASE CLASS

delete track only knows that track points to a Song, because that is the declared type of the pointer, and Song's destructor is not virtual. Exactly like the non-virtual describe function earlier in this unit, the compiler resolves the call to ~Song at compile time instead of dispatching to ~LiveRecording at run time. LiveRecording's destructor, and everything it was responsible for cleaning up, never runs.

The fix follows the same pattern used for describe: mark the base class's destructor virtual.

```
class Song {
protected:
    string title;
public:
    Song(string title) : title(title) {}
    virtual void describe() const {
        cout << "Song: " << title << endl;
    }
    virtual ~Song() {
        cout << "Song destroyed: " << title << endl;
    }
};
```

```

class LiveRecording : public Song {
private:
    int* attendeeIds;

public:
    LiveRecording(string title, int audienceSize)
        : Song(title), attendeeIds(new int[audienceSize]) { }

    void describe() const override {
        cout << "LiveRecording: " << title << endl;
    }

    ~LiveRecording()
    {
        delete[] attendeeIds;
        cout << "LiveRecording attendee records released" << endl;
    }
};

void retire(Song* track) {
    delete track;
}

```

```
int main()
{
    Song* show = new LiveRecording("Reverie", 1200);
    retire(show);

    return 0;
}
```

The program now prints:

```
LiveRecording attendee records released
Song destroyed: Reverie
```

With `~Song` declared virtual, delete track dispatches to `~LiveRecording` first, exactly as dynamic binding dispatches an ordinary virtual function call, and `~LiveRecording` runs to completion, in derived-to-base order, before `~Song` runs. This leads to a rule with no real exceptions: any class intended to be deleted through a pointer to one of its base classes must declare that base class's destructor virtual. A base class with no other virtual functions at all can still need this if objects of a derived class will ever be destroyed polymorphically.

OBJECT SLICING

Virtual functions dispatch correctly through *references* and *pointers*, but they cannot rescue a program from copying an object by value into a variable of its base type. A function that takes its parameter as a plain Song, rather than a const Song&, only ever receives a Song -- whatever was passed in is copied into exactly that much space, and nothing more.

```
class Song {
protected:
    string title;
public:
    Song(string title) : title(title) {}

    virtual void describe() const {
        cout << "Song: " << title << endl;
    }
};

class LiveRecording : public Song {
private:
    string venue;

public:
```

```

    LiveRecording(string title, string venue) : Song(title), venue(venue)
{}

    void describe() const override
    {
        cout << "LiveRecording: " << title << " at " << venue << endl;
    }
};

void showInfo(Song s) { // taken by value -- this is where slicing happens
    s.describe();
}

int main() {
    LiveRecording concert("Reverie", "Carnegie Hall");
    concert.describe(); // correct: LiveRecording version
    showInfo(concert);  // sliced: prints the Song version, venue is lost
    Song plain = concert; // slicing again, through copy-initialization
    plain.describe();
    return 0;
}

```

The program prints:

```
LiveRecording: Reverie at Carnegie Hall
```

```
Song: Reverie
```

```
Song: Reverie
```

concert.describe() correctly dispatches to LiveRecording's version, since concert is used through its own type. But showInfo(concert) copies concert into a parameter of type Song, and that copy has no venue field and no LiveRecording override to dispatch to -- it is, as far as the type system is concerned, an ordinary Song. The venue is not hidden or inaccessible; it was never copied in the first place. This is called object slicing: the derived part of an object is sliced off whenever it is copied into a variable, parameter, or return value of its base type. Song plain = concert produces the same result through ordinary copy-initialization, with no function call involved at all.

COMMON MISTAKE: PASSING POLYMORPHIC OBJECTS BY VALUE

Slicing produces no warning and no error -- the code compiles cleanly and runs without crashing, which is exactly what makes it dangerous. Once a class participates in an inheritance hierarchy and relies on virtual dispatch, it should almost always be passed and stored by reference or by pointer. Don't pass by value 😊

DESIGN GUIDELINES

Using inheritance well is a matter of judgment, guided by a small set of habits that keep a class hierarchy easy to extend and hard to misuse.

- Use public inheritance only for a genuine is-a relationship. If a class simply needs another as a component, prefer *composition* -- a has-a member -- over inheriting from it.
- Keep a base class's public interface stable. Every derived class, and every function written against the base class, depends on it.
- Prefer *protected* accessor functions over protected data members where practical, since protected data still exposes representation details to every future derived class.
- Give any class intended for polymorphic use a virtual destructor, even if it has no other virtual functions yet.
- Mark every intended override with the override keyword, so a signature mismatch becomes a compile error instead of a silent hiding bug.
- Never rely on a virtual function call inside a base class constructor or destructor to reach a derived class's override.
- Pass and store polymorphic objects by reference or pointer, never by value, once inheritance and virtual functions are in play.

SUMMARY

This unit extended the Song class into LiveRecording and Podcast using public inheritance, modeling the is-a relationship between a general track and its more specific forms. A derived class's constructor must explicitly initialize its base class whenever that base has no default constructor, and construction always runs base-to-derived while destruction always runs derived-to-base, regardless of how a constructor's initializer list is written. protected access lets a chosen family of classes share implementation details that remain hidden from the rest of the program.

Marking a function virtual switches it from static binding, resolved by the declared type of a reference or pointer, to dynamic binding, resolved by the object's real type at run time -- the mechanism that makes one interface usable correctly across many related classes. The same mechanism applies to destructors: a base class meant for polymorphic use needs a virtual destructor, or deleting a derived object through a base pointer silently skips its cleanup. Copying a derived object into a base-typed variable or parameter slices off everything the derived class added, with no warning at all, which is why polymorphic objects should be handled by reference or pointer rather than by value.

REVIEW QUESTIONS

1. What is the difference between an is-a relationship and a has-a relationship? Give one example of each using classes from this unit.
2. Why does LiveRecording's constructor need to call Song's constructor explicitly in its initializer list?
3. What compiler error appears if a derived class's constructor never initializes a base class that has no default constructor?
4. Explain the difference between private and protected access, specifically from the point of view of a derived class.
5. Why does code outside the class hierarchy still fail to compile when it tries to read a protected member?
6. In what order do a base class's and a derived class's constructors run? In what order do their destructors run?
7. Why does the order in which a constructor's initializer list is written not determine the actual initialization order?
8. What warning does the compiler give when an initializer list is written in a misleading order, and what does that warning actually mean?

9. Explain the difference between overriding a virtual function and hiding a non-virtual one.
10. Why did calling `describe()` through a `const Song&` parameter call `Song`'s version instead of `LiveRecording`'s, before `virtual` was introduced?
11. What does marking a function `virtual` change about how a call to that function is resolved?
12. Why does calling a virtual function from inside a base class's own constructor never reach a derived class's override?
13. What does the `override` keyword do, and why does it not change a program's behavior at run time?
14. Why did adding `override` to a mismatched function signature turn a silent bug into a compile error?
15. What warning does the compiler give when a polymorphic base class has a non-virtual destructor, and why is it justified?
16. Explain what happens, step by step, when `delete` is called on a `Song*` that actually points to a `LiveRecording`, once `~Song` is virtual.

17. What is object slicing, and why does it occur even though the source object still has its full derived type?
18. Why does passing a polymorphic object by value defeat the purpose of writing virtual functions for it?
19. List three general design guidelines from this unit that apply to using inheritance responsibly.

LOOKING AHEAD

Every class in this unit still had to provide a complete, usable describe function, including Song itself. The next unit introduces abstract base classes and pure virtual functions, which let a base class declare an interface -- a function every derived class must implement -- without providing any implementation of its own. This makes it possible to write a base class that can never be instantiated directly, only extended, and to store a genuinely mixed collection of related objects, such as a vector of Song pointers holding live recordings and podcasts side by side, and process every one of them correctly through a single shared interface. This is the foundation of runtime polymorphism that this unit's virtual functions and virtual destructors were building toward.